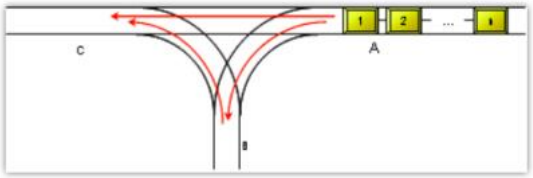
Hình 13 là cơ cấu đường tàu tại một ga xe lửa



Di chuyển toa tàu

Ban đầu ở đường ray A chứa các toa tàu đánh số từ 1 tới n theo thứ tự từ trái qua phải, người ta muốn chuyển các toa đó sang đường ray C để được một thứ tự mới là một hoán vị của (1, 2, ..., n) theo quy tắc: chỉ được đưa các toa tàu chạy theo đường ray theo hướng mũi tên, có thể dùng đoạn đường ray B để chứa tạm các toa tàu trong quá trình di chuyển.

a. Hãy nhập vào hoán vị cần có, cho biết có phương án chuyển hay không, và nếu có hãy đưa ra cách chuyển:

Ví dụ: n = 4 thì thứ tự cần có là (1, 4, 3, 2)

1)A → C; 2)A → B; 3)A → B; 4)A → C; 5)B → C; 6)B → C

b. Những hoán vị nào của thứ tự các toa là có thể tạo thành trên đoạn đường ray C với luật di chuyển như trên.

a/ Dùng stack:

```
def check_stack_simulation(permutation):
    stack = []
    next_train = 1
    n = len(permutation)
    steps = []  # lưu lại các bước di chuyển
    for want in permutation:
        while next_train <= n and (not stack or stack[-1] != want):
            steps.append(f"A → B ({next_train})")
            stack.append(next_train)
            next_train += 1
        if stack and stack[-1] == want:
            steps.append(f"B → C ({stack[-1]})")
            stack.pop()
        else:
            return False, steps
    return True, steps
```

b/ Dùng hàng đợi:

```
from collections import deque
def check_queue_simulation(permutation):
    queue = deque()
    next_train = 1
    n = len(permutation)
    steps = []
    for want in permutation:
        while next_train <= n and (not queue or queue[0] != want):
            steps.append(f"A → B ({next_train})")
            queue.append(next_train)
            next_train += 1
        if queue and queue[0] == want:
            steps.append(f"B → C ({queue.popleft()})")
```

```
        else:
            return False, steps
    return True, steps
```

```
Code chạy:
perm = [1, 4, 3, 2]
ok_stack, steps_stack = check_stack_simulation(perm)
ok_queue, steps_queue = check_queue_simulation(perm)
print("== STACK ==")
print("Hợp lệ:", ok_stack)
print("Các bước:")
for s in steps_stack:
    print(" ", s)
print("\n== QUEUE ==")
print("Hợp lệ:", ok_queue)
print("Các bước:")
for s in steps_queue:
    print(" ", s)
```

b/

Những hoán vị nào của thứ tự các toa là có thể tạo thành trên ray C là tập hợp các hoán vị không chứa mẫu 2-3-1, hay nói cách khác. Là các hoán vị có thể sinh ra từ thao tác push-pop của stack, với số lượng là Catalan số:

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

```
from itertools import permutations

def is_stack_permutation(p):
    stack = []
    next_train = 1
    n = len(p)
    for want in p:
        while next_train <= n and (not stack or stack[-1] != want):
            stack.append(next_train)
            next_train += 1
        if stack and stack[-1] == want:
            stack.pop()
        else:
```

```
            return False
    return True
def list_valid_stack_permutations(n):
    all_perms = permutations(range(1, n+1))
    valid_perms = [p for p in all_perms if is_stack_permutation(p)]
    return valid_perms
# ✅ In kết quả cho n = 3, 4, 5
for n in [3, 4, 5]:
    print(f"\nCác hoán vị hợp lệ với n = {n}:")
    valid = list_valid_stack_permutations(n)
    for p in valid:
        print(p)
    print(f"→ Có {len(valid)} hoán vị hợp lệ (theo Catalan): C_{n} = {len(valid)}")
```